Lecture 41 Practice Solutions (Hinglish): Deep Learning (Part-6)

Complete Mathematical Derivations & Code Solutions

Solution 1: Exploratory Data Analysis & Deduplication

1. **Data Leakage:** Jab train aur test set divide karne se pehle duplicate samples delete nahi kiye jaate, toh wahi same review training me bhi chala jata hai aur testing me bhi. Isse model real generalization sikhne ke badle us review ko memorize kar leta hai aur test accuracy artificially high dikhane lagti hai.
2. **Numbers:** $50,000 - 418 = \mathbf{49,582}$ clean reviews. Negative = $24,698$ (49.8%), Positive = $24,884$ (50.2%). Class balance perfectly $50/50$ maintained rehta hai.

Solution 2: Lowercasing ka Importance

1. Normalization se har letter standard lowercase ban jata hai taaki model capital aur small letters me confuse na ho.
2. `"Good"` aur `"GOOD"` bina lowercasing ke do alag words maane jayenge (count 1 each). Lowercase karne par ek single token `"good"` banega jiska count 2 ho jayega, jisse feature signal double strong ho jayega.

Solution 3: Regex URL Removal

1. `http` : "http" aur "https" match karta hai. `\S` : Non-whitespace characters (letters, numbers, slashes). `+` : One or more characters jab tak agla space na mile.
2. `.*` greedy hota hai jo line ke end tak sab kuch match kar leta hai. Agar `http.*` use karein toh URL ke baad likha hua saara review text delete ho jayega!

Solution 4: Regex Greedy vs Non-Greedy

Sentence: `"<br />This was an incredible movie!<br />Loved the plot."`

1. Non-greedy (`<.*?>`): Har tag alag se match hota hai aur sirf `<br />` hat-ta hai. Output: `"This was an incredible movie!Loved the plot."`

2. Greedy (`<.*>`): Pehle `<` se aakhiri `>` tak pura beech ka sentence (`<br />This was an incredible movie!<br />`) ek saath delete ho jata hai! Sirf `"Loved the plot."` bachega.

Solution 5: Caret Negation

1. `[...]` ke andar caret `^` lagane se negation ho jata hai (jo set me nahi hai use match karo).
2. Sentence `"Wow!! The actor's performance was 10/10 #CinematicMasterpiece :)"` par apply karne se: Result: `"Wow The actors performance was 1010 CinematicMasterpiece "`

Solution 6: NLTK word_tokenize vs .split()

1. Python `.split(" ")` punctuations ko words ke sath chipka deta hai (`"movie!"`) aur contractions (`"didn't"`) ko alag nahi kar pata.
2. NLTK `word_tokenize()` `"didn't"` ko `["did", "n't"]` aur `"movie!"` ko `["movie", "!"]` me properly separate karta hai.

Solution 7: Stopwords Removal aur Zipf's Law

1. Zipf's Law: $f(w) \propto 1/\text{rank}(w)$. Top 100 English words ("the", "is", "at") natural text ka lagbhag 50% space cover karte hain.
2. Stopwords har review me equally aate hain chahe film achhi ho ya buri. Unko delete karne se text length 50% kam ho jati hai aur model ka pura dhyan important sentiment words par rehta hai.

Solution 8: Stemming vs Lemmatization

1. **Stemming (Porter):** Rule-based suffix cutting (fast, aksar invalid dictionary words banata hai jaise `"techniq"`). **Lemmatization (WordNet):** POS grammar + dictionary lookup (slow, hamesha real valid root words banata hai jaise `"technique"`).
2. **Over-stemming:** `"universal"` aur `"university"` dono ko `"univers"` bana dena. **Under-stemming:** `"alumnus"` aur `"alumni"` alag reh jana.
3. 50,000 reviews me 15 lakh words hain. Lemmatization 15 minute leti jabki Porter Stemmer 45 second me complete ho gaya aur sentiment classification ke liye perfectly accurate tha.

Solution 9: TF-IDF aur max_features=5000

1. $49,582 \times 60,000 \times 4$ bytes $\approx \mathbf{11.1}$ GB RAM (system crash ho jayega).
   $49,582 \times 5,000 \times 4$ bytes $\approx \mathbf{945}$ MB RAM (92% memory bachat!).
2. Jab word har review me ho ($\text{DF} = N$):

$$\text{IDF} = \log\left(\frac{1+N}{1+N}\right) + 1 = \log(1) + 1 = \mathbf{1.0}$$

   Uska weight minimum 1.0 par suppress ho jata hai.

Solution 10: DataLoader aur Shuffling

1. Total batches = $\lceil 39,665/64 \rceil = \mathbf{620}$ batches. Aakhiri batch me $39,665 - (619 \times 64) = \mathbf{49}$ samples honge.
2. Agar `shuffle=False` ho, toh mini-batches correlated ban jaate hain aur gradient descent zigzag karke galat local minima me atak sakta hai.

Solution 11: 3D Input Tensor Reshaping

1. `(64, 5000)` ko `Xb.unsqueeze(1)` karne par shape $(\mathbf{64}, \mathbf{1}, \mathbf{5000})$ banta hai.
2. `Batch = 64`, `Seq_Len = 1` (pura review 1 timestep me feed ho raha hai), `Input_Dim = 5000` (TF-IDF features).

Solution 12: Parameter Count Calculation

1. `self.rnn`:
   - W_{xh} : $128 \times 5000 = 640,000$
   - W_{hh} : $128 \times 128 = 16,384$
   - b_h : 128 Subtotal = $656,512$
2. `self.fc`:
   - W_{hy} : $1 \times 128 = 128$
   - b_y : 1 Subtotal = 129
3. Total = $656,512 + 129 = \mathbf{656,641}$ parameters.

Solution 13: Binary Cross-Entropy Loss

1. Formula: $\mathcal{L} = -\frac{1}{B}\sum[y\log(\hat{p}) + (1-y)\log(1-\hat{p})]$.
2. $y = 1$ par:

- $\hat{p} = 0.99 \implies -\ln(0.99) \approx \mathbf{0.010}$
- $\hat{p} = 0.01 \implies -\ln(0.01) \approx \mathbf{4.605}$ (450 times zyada loss!). Galat prediction par bohot bhaari penalty lagti hai.

Solution 14: Model Evaluation Protocol

1. `model.eval()` se Dropout OFF ho jata hai aur BatchNorm statistics update hona band ho jaati hain.
2. `torch.no_grad()` autograd tracking band karta hai jisse $> 60\%$ memory bachti hai aur testing $2\times$ fast ho jaati hai.
3. Logit $z = -1.386$:

$$\hat{p} = \sigma(-1.386) = \frac{1}{1 + e^{1.386}} \approx \mathbf{0.20}$$

Kyuki $0.20 \leq 0.5$, predicted class **Negative [0]** hogi.

Solution 15: CNN vs RNN on MNIST (28×28)

1. MNIST image ko RNN me feed karne ke liye 28 rows ko 28 sequential timesteps maana jata hai ($T = 28, d_x = 28$). Step 1 par Row 1 scan hoti hai, step 28 par Row 28 scan hoti hai.
2. CNN 2D spatial patches (3×3) ko horizontally aur vertically dono taraf ek sath scan karta hai aur translation invariance use karta hai, isliye $> 99.2\%$ accuracy deta hai. RNN 1D top-to-bottom scan karta hai jisse vertical dependencies weak pad jaati hain, isliye accuracy $\approx 97\%$ rehti hai.